

Planificación Técnica de la Solución

Actividad N°: 9 **Fecha:** 11/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Definir la arquitectura tecnológica de la solución (stack, componentes, justificación de decisiones) y planificar técnicamente cómo se construirá el sistema durante las próximas semanas.

2. Stack tecnológico seleccionado

Backend

Tecnología	Uso
Node.js + Express	Servidor y API REST
MongoDB (Atlas) + Mongoose	Base de datos y modelado de esquemas
JSON Web Token (jsonwebtoken)	Autenticación de sesión
bcryptjs	Hash de contraseñas
Socket.IO	Comunicación en tiempo real
Helmet, express-rate-limit, cors	Seguridad de la API
Winston	Logging estructurado
express-validator	Validación de datos de entrada
csv-parser	Importación del archivo de venta

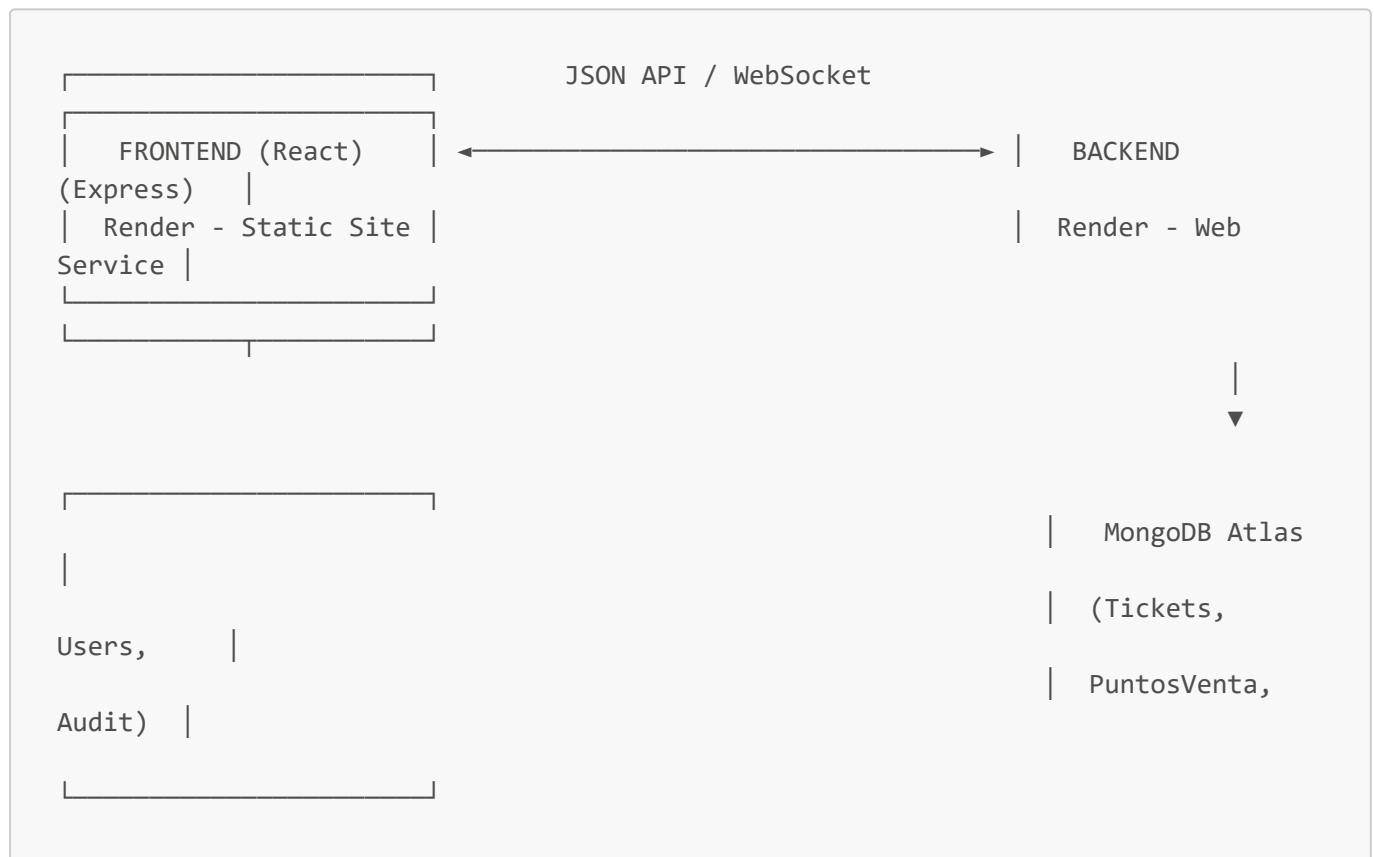
Frontend

Tecnología	Uso
React 18 + Vite	Interfaz de usuario y bundling
React Router v6	Enrutamiento entre páginas
React Bootstrap + Bootstrap 5	Componentes visuales
Context API	Estado global de autenticación
Axios	Consumo de la API REST
Socket.IO Client	Recepción de actualizaciones en tiempo real
Chart.js / react-chartjs-2	Gráficos del dashboard
SweetAlert2	Alertas y confirmaciones al usuario

3. Justificación de decisiones clave

- **MongoDB sobre SQL:** la estructura de un boleto varía poco pero el volumen y la necesidad de importar datos semi-estructurados de un CSV (columnas variables por evento) encaja mejor con un modelo de documentos flexible.
- **JWT sobre sesiones de servidor:** al desplegar backend y frontend como servicios independientes en Render (dominios distintos), un token portátil evita depender de cookies de sesión compartidas y simplifica el escalado horizontal.
- **Socket.IO sobre polling:** el requerimiento de "ver el cambio al instante entre varios puntos de venta" (evitar doble canje por desincronización) exige push en tiempo real; el polling periódico generaba carga innecesaria y latencia de varios segundos.
- **Arquitectura de colección única y localidades dinámicas:** al atender un evento a la vez con localidades que cambian de concierto en concierto, fijar las localidades en el código obligaría a un despliegue de código por cada evento; extraerlas del CSV en tiempo de importación elimina ese mantenimiento.

4. Arquitectura general de la solución



5. Plan de despliegue

- **Backend:** servicio web en Render, build `npm install`, start `npm start`, variables de entorno gestionadas desde el panel de Render (`MONGODB_URI`, `JWT_SECRET`, `CORS_ORIGIN`, etc.), nunca committeadas en texto plano en el repositorio.
- **Frontend:** sitio estático en Render, build `npm run build`, `VITE_API_URL` apuntando al backend desplegado; reglas de rewrite para servir la SPA (`index.html`) en todas las rutas.
- **Base de datos:** MongoDB Atlas, accesible por URI segura, con whitelist de IPs configurada para los servicios de Render.

6. Planificación de sprints (visión general)

Semana	Enfoque
3	Diseño de arquitectura, base de datos y componentes
4	Autenticación, gestión de usuarios y control de acceso
5	Funcionalidades de gestión, consulta, validación y canje
6	Control, seguimiento y consulta (auditoría, dashboard)
7	Integración y pruebas
8	Documentación y cierre

7. Conclusiones del día

Queda definida la arquitectura técnica completa de la solución y su justificación, lista para iniciar el diseño detallado en la Semana 3, así como el plan de despliegue en la nube.